Rule Engine Comma Separated Enhancement

Knowledge Representation & Reasoning (KRR)

> **Core Insight:** KRR is the heart of Artificial Intelligence. Even biological approaches like neural networks build upon KRR theory. Essential reading includes Brachman & Levesque's *"Knowledge Representation and Reasoning"* and Russell & Norvig's *"AI: A Modern Approach."*

---

1. Understanding KRR

KRR focuses on representing knowledge in **symbolic form** and simulating the act of thinking using that knowledge. While object-oriented languages use classes (like Java Beans) to describe entities, researchers use more sophisticated models like **OWL (Web Ontology Language)** to represent complex world relationships.

Simple Example

Imagine you have data like:

- "Ahmed is a developer"
- "Developers write code"

With OWL, you can formally define:

- Classes: `Person`, `Developer`
- Relationships: `writesCode`
- Rules: "Every Developer is a Person"

Now a system can **infer**:

👉 Ahmed is a Person

👉 Ahmed writes code

---

2. Production Rule Systems (PRS)

A rule engine delivers KRR functionality to developers through three primary components:

Component	Description	Example
Ontology	The representation model	Java classes, OWL, records
Rules	Perform reasoning/facilitate "thinking"	`when <conditions> then <actions>`
Data	Facts asserted into memory	Instances of a class

- Drools started life as a specific type of rule engine called a Production Rule System (PRS) and was based around the Rete algorithm. The Rete algorithm, developed by Charles Forgy in 1974, forms the brain of a Production Rule System and is able to scale to a large number of rules and facts. A Production Rule is a two-part structure: the engine matches facts and data against Production Rules - also called Productions or just Rules - to infer conclusions which result in actions.

```
when
    <conditions>
then
    <actions>;
```

- The process of matching the new or existing facts against Production Rules is called *pattern matching*, which is performed by the *inference engine*. Actions execute in response to changes in data, like a database trigger; we say this is a *data driven* approach to reasoning. The actions themselves can change data, which in turn could match against other rules causing them to fire; this is referred to as forward chaining
- The Rules are stored in the Production Memory and the facts that the Inference Engine matches against are kept in the Working Memory. Facts are asserted into the Working Memory where they may then be modified or retracted. A system with a large number of rules and facts may result in many rules being true for the same fact assertion; these rules are said to be in conflict. The Agenda manages the execution order of these conflicting rules using a Conflict Resolution strategy.

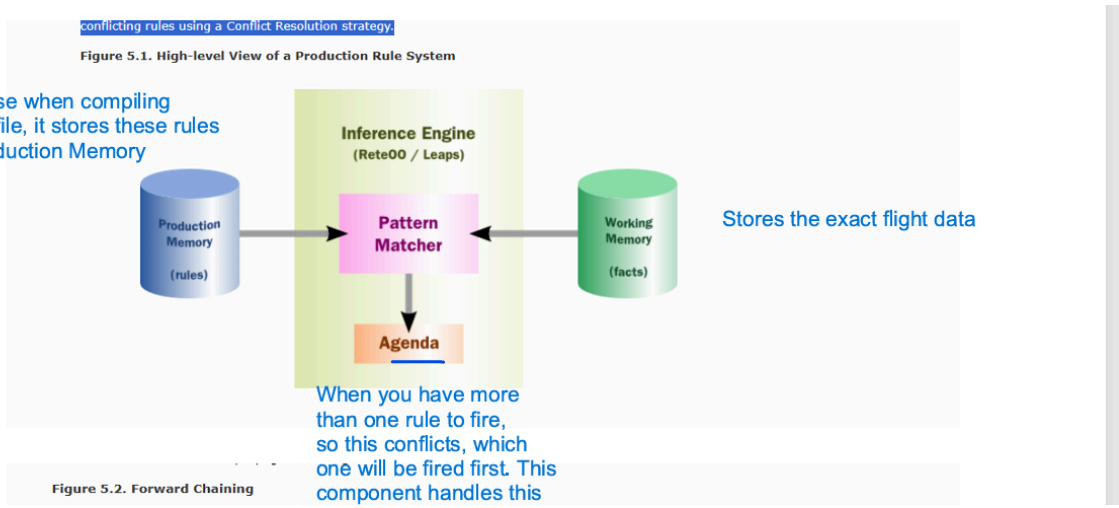
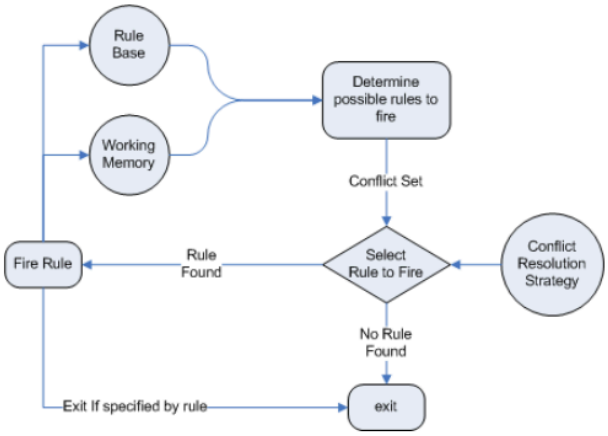


Figure 5.2. Forward Chaining

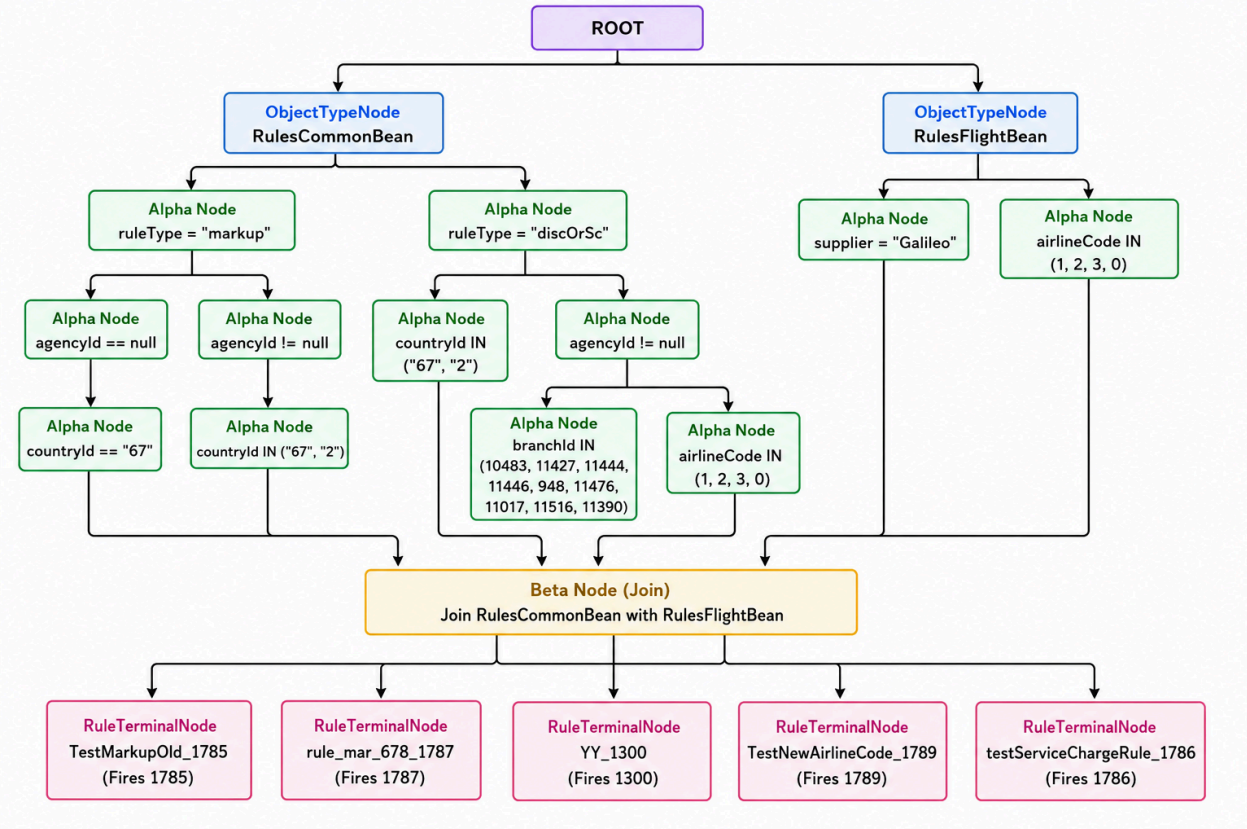


3. The Rete Algorithm: Under the Hood

- The algorithm processes rules into a **discrimination network** and filters data as it propagates through these nodes.
- The compilation algorithm describes how the Rules in the Production Memory are processed to generate an efficient discrimination network. In non-technical terms, a discrimination network is used to filter data as it propagates through the network. The nodes at the top of the network would have many matches, and as we go down the network, there would be fewer matches.

Node Types & Roles

Node Type	Role	Key Behavior
Root	Entry Point	All objects enter here.
ObjectType	Type Filtering	Routes objects by class (via `instanceof` check).
AlphaNode	Literal Conditions	Evaluates single-field literal constraints. **Hashed for O(1) lookup.**
BetaNode	Cross-object Joins	Compares two objects (JoinNode/NotNode).
TerminalNode	Rule Match	Indicates all conditions are met; the rule is ready to fire.



- **Working Memory** is the runtime container — every `insert()`, `update()`, and `retract()` call originates here and propagates down the network.

- **ObjectTypeNode (OTN)** is the first gate. It answers *"what type is this fact?"* and routes it to the correct sub-network. There is exactly one OTN per Java class, shared by all rules that use that type. It also understands inheritance — inserting a `Dog` fact propagates through both the `Dog` OTN and the `Animal` OTN.
- **AlphaNodes** filter individual facts using field-level constraints (`status == "ACTIVE"`, `age > 18`). They operate on a *single object* in isolation. The key optimization here is **sharing** — if two rules both check `status == "ACTIVE"` on the same type, that AlphaNode is created once and reused by both.
- **BetaNodes** are where things get more powerful — they join *two inputs* together (left memory + right memory).

Node Sharing

To save memory and compute, Drools "collapses" identical patterns. If two rules check for Account `(name == "omar")`, they share the same **AlphaNode**, splitting only when their specific logic (BetaNodes) differs.

---

4. Optimization: AlphaNode Hashing

Drools optimizes the path from `ObjectTypeNode` to `AlphaNode` using a **HashMap**. Instead of checking every literal constraint sequentially, Drools uses the literal value as a key to jump directly to the correct node.

The Problem: Opaque Constraints

Any constraint that is not a plain `field == "literal"` (e.g., method calls, `eval()` blocks, or `new` expressions) **bypasses the HashMap**. This forces the engine into a sequential scan, drastically slowing performance.

Side-by-Side Comparison

Strategy	DRL Constraint Example	Performance
Avoid (Opaque)	`RulesFlightBean(new HashSet(Arrays.asList("Galileo")).contains(supplier))`	**O(n) Traversal:** Every fact tested individually.
Avoid (Eval)	`eval(supplier.equals("Galileo"))`	**O(n) Traversal:** `eval` is

		opaque to the compiler.
Use (Direct)	`RulesFlightBean(supplier == "Galileo")`	**O(1) Hash Lookup:** Direct retrieval.

What is doing now in Drool file?

- AirlineCodes or any other string values like supplier , stored like

```
$RulesFlightBean:RulesFlightBean((new HashSet(Arrays.asList( "Galileo")).contains(supplier)))
```

- This translated in Drool file into a list not hashmap or hashset
- How two strings be compared in Java?

```
// Place 1 – the Heap (normal object creation)
String s1 = new String("Galileo");   // new object on heap → address 0x4A2F
String s2 = new String("Galileo");   // another new object → address 0x7B1C
// s1 and s2 have the same content but different memory addresses

// Place 2 – the String Pool (interned)
String s3 = "Galileo";               // goes to string pool → address 0x1C3B
String s4 = "Galileo";               // same pool entry     → address 0x1C3B
// s3 and s4 point to the exact same memory address
```

- In Case-1
  - the comparison here must be S1.equals(S2) and it will take O(m) as both strings will be stored in different addresses in Heap , so S1 == S2 will not work.

- BUT in Case-2
  - Both S1 & S2 will point to the same memory address in String pool, so the comparison in this case will be between two addresses so it will take O(1), as you can compare like

    S1 == S2, no need to compare S1.equals(S2) which will take O(m), m is the length of strings
- So when KieSession fires rule , it takes O(n) to fetch the location of the target value in that list , and will take O(m) to make the comparison between two strings
- So Total complexity in the current version is O(n * m) → Takes much time

Approach -1: Use AlphaNode to make comparison

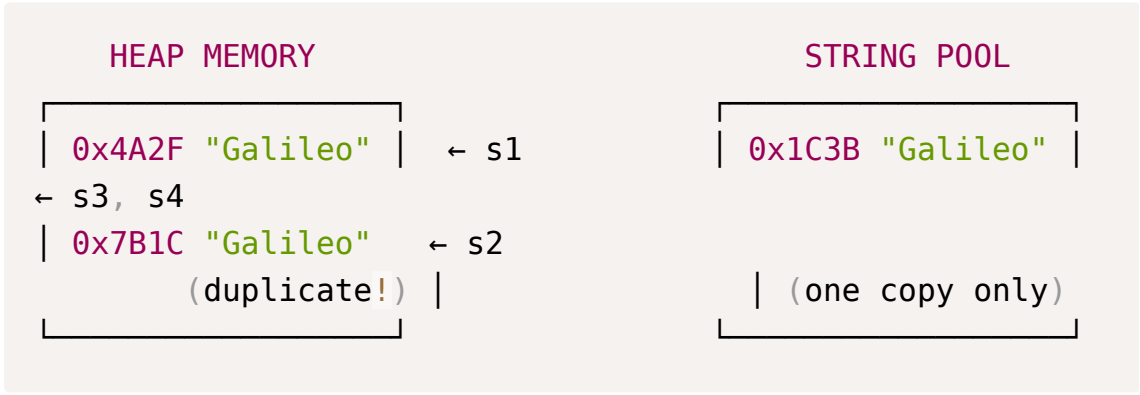
- AlphaNode holds the value, and be stored in HashMap in Production Memory
- So let's say we have values like (Galileo, Amadeus), and a condition like (supplier == "Galileo" || supplier == "Amadeus")
- This telling Rule Engine to store these values , each one in different alphaNode, and these nodes will be stored in for for HashTable.
- So when comparing (supplier == "Galileo"), it will fetch the value of "Galileo" from this HashTable by the alphaNode address which will take O(1) to fetch this value
- BUT it will take O(m) to compare supplier == "Galileo", so Total complexity will be O(1 * m) → O(m)
- So we get rid of the O(n) complexity for fetching the value like the old version
- Drool will be like
  - ```
    // Optimized for O(1) AlphaNode Hashing
    RulesFlightBean(
        supplier == "Galileo" ||
        supplier == "Sabre" ||
        supplier == "Amadeus"
    )
    ```

Approach-2: String Interning
    ```

- What is the meaning of String Interning?
  - When you create a string, the JVM can store it in **two different places** in memory.

```
// Place 1 — the Heap (normal object creation)
String s1 = new String("Galileo");   // new object on hea
p → address 0x4A2F
String s2 = new String("Galileo");   // another new objec
t → address 0x7B1C
// s1 and s2 have the same content but different memory a
ddresses

// Place 2 — the String Pool (interned)
String s3 = "Galileo";               // goes to string po
ol → address 0x1C3B
String s4 = "Galileo";               // same pool entry
→ address 0x1C3B
// s3 and s4 point to the exact same memory address
```



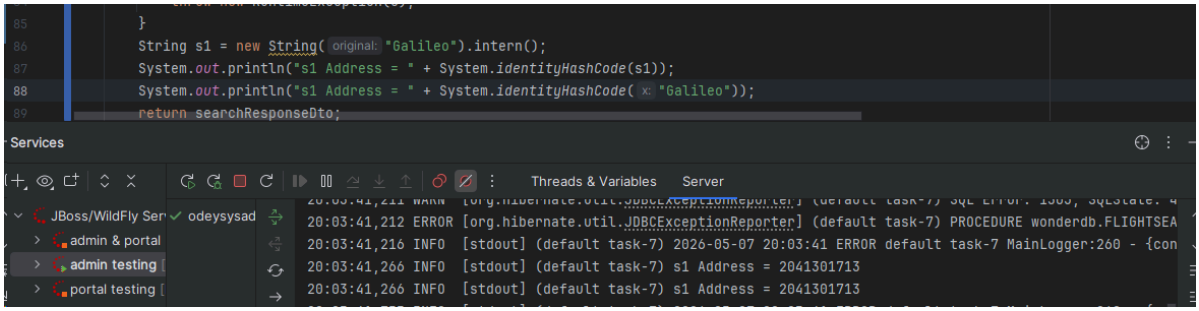

```
// == compares MEMORY ADDRESSES (pointer equality)
s1 == s2   // false — different heap addresses, even thou
gh same content
s3 == s4   // true  — same pool address
s1 == s3   // false — heap address vs pool address
// .equals() compares CHARACTER BY CHARACTER
s1.equals(s2)  // true — same content, but slower (char-b
```

```
y-char scan)
s1.equals(s3)  // true — same content
```

```
String s1 = new String("Galileo");  // on heap → 0x4A2F
String s2 = s1.intern();            // moves to pool → 0x
1C3B
// Now:
s2 == "Galileo"   // true! — both point to pool address 0
x1C3B
s1 == "Galileo"   // false — s1 still on heap at 0x4A2F
```

- `.intern()` says: *"Look up this string's value in the pool. If it exists, return the pool reference. If not, add it and return the pool reference."*
- Why this matters for Drools specifically
  - String literals you write in your `.drl` file — like `"Galileo"` — are **automatically interned** by the JVM at class load time. They live in the string pool.
  - But string values coming from your **RulesFlightBean is** typically created with `new String(...)` — live on the heap with new address.

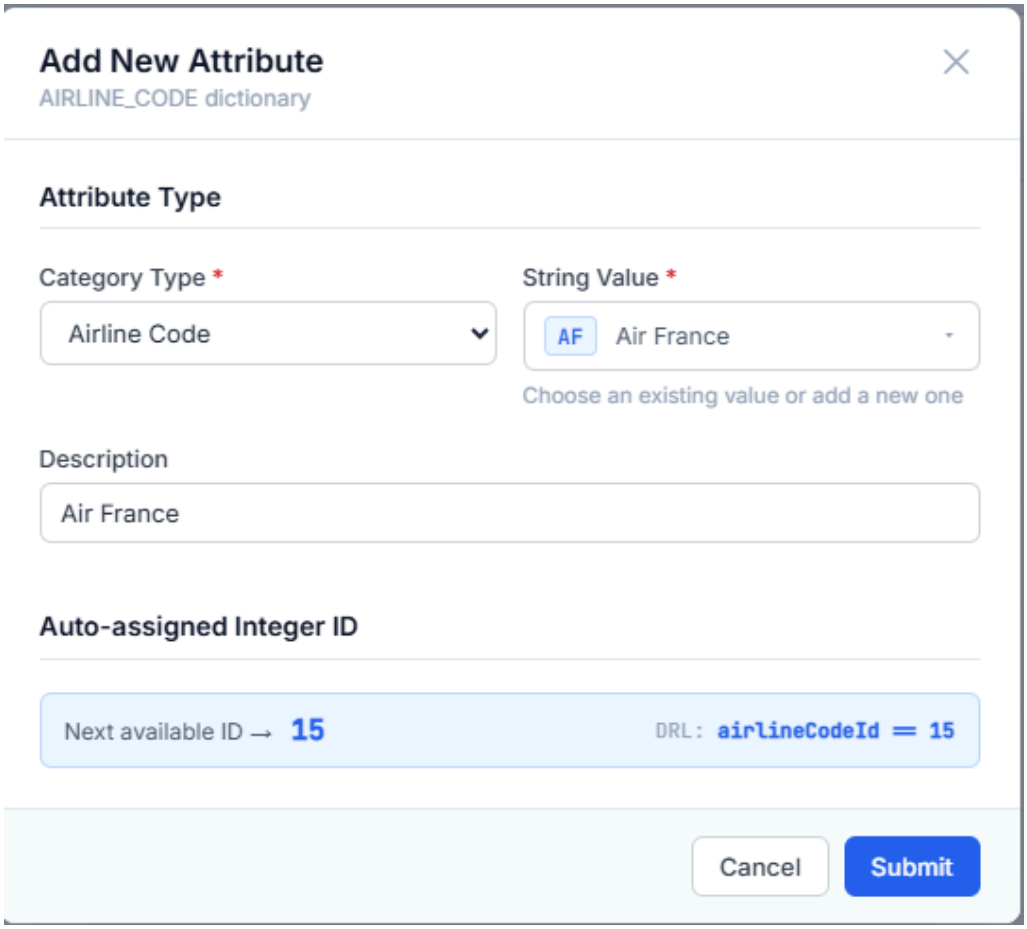
- So when supplier == “Galileo”, KieSession will take O(1) to check both addresses point to the same value in string pool or not



- As you can see both s1 variable and “Galileo” value , both have the same hashCode which in Java refers to the address in String pool

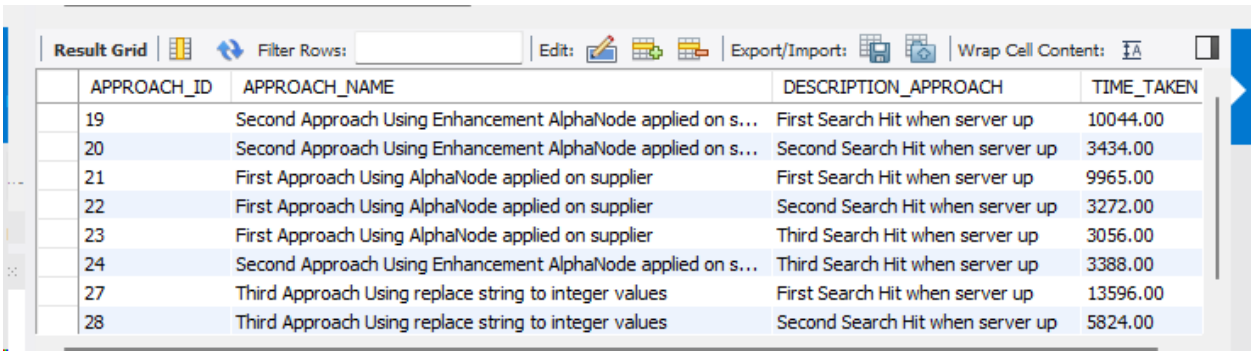
Approach-3: Add CRUD To Convert String values into Integers values

- Add a generic CRUD for all string attributes , which forcing the user to enters airlineCode or any other string value using integers values, let's say IDs.
- So Drool will treat with integers not string values



- But there will be a mapping from string values into integers values twice
- One when adding these values into Drool file and second one when firing rule to convert the airlineCode come from supplier (FlightOption) into integer value

Approaches Result



Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

APPROACH_ID	APPROACH_NAME	DESCRIPTION_APPROACH	TIME_TAKEN
19	Second Approach Using Enhancement AlphaNode applied on s...	First Search Hit when server up	10044.00
20	Second Approach Using Enhancement AlphaNode applied on s...	Second Search Hit when server up	3434.00
21	First Approach Using AlphaNode applied on supplier	First Search Hit when server up	9965.00
22	First Approach Using AlphaNode applied on supplier	Second Search Hit when server up	3272.00
23	First Approach Using AlphaNode applied on supplier	Third Search Hit when server up	3056.00
24	Second Approach Using Enhancement AlphaNode applied on s...	Third Search Hit when server up	3388.00
27	Third Approach Using replace string to integer values	First Search Hit when server up	13596.00
28	Third Approach Using replace string to integer values	Second Search Hit when server up	5824.00

- In this screen shot, I applied the new approaches on both supplier & airlineCode
- Notice that the first approach is better than the other approaches
- They are close in results but the first one is better , why despite the enhancement did in the first approach using intern() and third approach use integers values?

How intern() function works?

- **Pool Lookup**: It checks the String Pool to see if a string with the exact same character sequence already exists.
- **Existing Case**: If the string is already in the pool, the JVM simply returns a reference to that existing pool instance.
- **New Case**: If it does not exist, the JVM adds the current string object to the pool and returns a reference to it

Performance vs. Memory Trade-off

- **Pros**:
  - **Memory Efficiency**: Significant savings when your application has many duplicate strings.
  - **Faster Comparisons**: Once interned, you can compare strings using the `==` operator (identity comparison) rather than `.equals()`, which is significantly faster.
- **Cons**:
  - **CPU Overhead**: The lookup and insertion process into the pool takes time.

- **Unbound Growth**: Interning "one-off" strings that won't be repeated can lead to unnecessary memory consumption in the pool

Why first approach better than other approaches?

- AirlineCodes are stored as string in DB
- When user creates a rule, he enters the airlineCode value as string so there is a need to map the values stored in DB specially in TT_TS_RULE_CRITERIA_ELEMENT table from string to integer
- This mapping is happened twice, one when adding these values into Drool file and second one when firing rule to convert the airlineCode come from supplier (FlightOption) into integer value
- This performance issue can be fixed when forcing the user to enter an integers value when creating a rule, so by this we'll get rid of the first mapping (which takes the much time)